Data Extraction Easy With jssdataloom

Many companies do data extraction to CSV(comma or pipe separated) files, because they have tools to import data.

jssdataloom is based on JSS(https://github.com/LaranIkal/jss) - GraalVM Polyglot Context JavaScript shell. jssdataloom is created to extract data using JavaScript and Java JDBC libraries, so you can:

- Extract data from a database such as MySQL, PostgreSQL, Oracle, or others.
- Transform the data into a specific format.
- Save it to a CSV file, or pipe separated file, define it in file: config.jss

Installation:

- Install Java from graalvm(https://www.graalvm.org/), Go to https://github.com/graalvm/graalvm-ce-builds/releases/. Minumum Java JDK version Needed : graalvm-community-25.4.4.1.1+1.1
- **On Linux:** Open graalvm-community-jdk-25i4-25.0.4.1.1_linux-x64_bin.tar.gz
- **On Windows:** Open graalvm-community-jdk-25i4-25.0.4.1.1_windows-x64_bin.zip

Extract folder graalvm-community-25.4.4.1.1+1.1 inside folder $HOME/Apps on Linux or C:\Apps on Windows.

You will have this folder $HOME/Apps/graalvm-community-25.4.4.1.1+1.1 or C:\Apps\graalvm-community-25.4.4.1.1+1.1

To keep the path short, rename folder:

graalvm-community-25.4.4.1.1+1.1 to graalvmjdk25, so you will have:

$HOME/Apps/graalvmjdk25 or C:\Apps\graalvmjdk25

Extract the script engine and scripts, folder jssdataloom to $HOME/Apps on Linux or C:\Apps on Windows

You must have folder: $HOME/Apps/jssdataloom or C:\Apps\jssdataloom

Now you can run the batch files inside folder:

- Linux -> $HOME/Apps/jssdataloom/runscripts
  - or
- Windows -> C:\Apps\jssdataloom\runscripts

Folders structure:

```
📁 jss (JavaScript PolyglotContext shell), this is the root folder.
├── jss-1.2.jar                # JavaScript PolyglotContext shell, fat jar
├── jssrepl.jar                # JavaScript PolyglotContext interactive shell, fat jar
├──📁runscripts
│  ├── cleanfile.bat           # Starts utils/cleanfile.jss on Windows
│  ├── cleanfile.sh            # Starts utils/cleanfile.jss on Linux
│  ├── dataextraction.bat      # Shell file for Linux to start the extract JavaScript script
│  ├── dataextraction.sh       # Batch file for Windows to start the extract JavaScript script
│  ├── getjarlib.bat           # Starts utils/getjarlib.jss on Windows
│  ├── getjarlib.sh            # Starts utils/getjarlib.jss on Linux
│  ├── jssrepl.bat             # jssrepl launcher on Windows
│  └── jssrepl.sh              # jssrepl launcher on Linux
└──📁scripts
   ├── config.jss                    # Global configuration
   ├──📁lib
   │  ├── GeneralLibs.jss            # Entry point — loads all libs + utility functions
   │  ├── DataBaseConnections.jss    # utils.getDBConnection()
   │  └── DataIndexer.jss            # indexer.* functions
   ├──📁dataextraction
   │  ├── dataextraction.jss         # Main extraction script
   │  ├── dataextractionqueries.jss  # SQL queries
   │  ├── filtersandxref.txt         # Filter/Xref data (pipe-delimited)
   │  ├── filtersandxrefindexed.json # (generated) Indexed structure for debugging
   │  ├──📁datafiles
   │  │  ├── InventoryTransactions.csv   # (generated) Output with xref applied
   │  │  └── NoXrefInventoryTransactions.txt  # (generated) Rows that failed xref
   │  ├──📁docs
   │  │  └── Data Extraction Layout And Description.xlsx
   │  └──📁log
   │     └── dataextraction.log
   └──📁utils
      ├── cleanfile.jss # Reads an input file and cleans all non printing characters.
      └── getjarlib.jss # This script helps you to download all the Java JDBC libraries needed.
```